

```

#include <iostream>
#include <cstdlib>
#include <ctime>

using namespace std;

#define SIZE 10
#define UP 0
#define DOWN 1
#define LEFT 2
#define RIGHT 3

class Grid {
public:
    Grid() {
        Grid(0);
    }
    Grid(int s) {
        state = s;
        dir[UP] = NULL;
        dir[DOWN] = NULL;
        dir[LEFT] = NULL;
        dir[RIGHT] = NULL;
    }
    Grid* getDir(int d) { return dir[d]; }
    int getState() { return state; }
    void setDir(int d, Grid* g) { dir[d] = g; }
    void setState(int s) { state = s; }
private:
    Grid* dir[4];
    int state;
};

struct List {
public:
    List() {
        top = 0;
    }
    void addElement(Grid* g) {
        if (top < SIZE * SIZE) {
            data[top++] = g;
        }
    }
    Grid* removeElement() {
        if (top > 0) {
            return data[--top]; // Return the last element and decrease top
        }
        return nullptr; // Return NULL if list is empty
    }
    void printPath() {
        int j;
        for (j = 1; j < top; j++) {
            if (data[j] == data[j - 1]->getDir(UP)) {
                cout << "UP\n";
            }
            else if (data[j] == data[j - 1]->getDir(DOWN)) {
                cout << "DOWN\n";
            }
            else if (data[j] == data[j - 1]->getDir(LEFT)) {
                cout << "LEFT\n";
            }
        }
    }
};

```

```

        }
        else if (data[j] == data[j - 1]->getDir(RIGHT)) {
            cout << "RIGHT\n";
        }
    }
}
private:
    Grid* data[SIZE * SIZE];
    int top;
};

class Maze {
public:
    Maze() {
        initMaze(SIZE);
    }

    void initMaze(int s) {
        // Create a 2D array of Grid pointers
        maze = new Grid**[s];
        for (int i = 0; i < s; i++) {
            maze[i] = new Grid*[s];
        }

        // Initialize all grids to empty (state 0)
        for (int i = 0; i < s; i++) {
            for (int j = 0; j < s; j++) {
                maze[i][j] = new Grid(0);
            }
        }

        // Randomly generate 20% walls
        srand(time(0));
        int wallCount = (s * s) / 5;
        while (wallCount > 0) {
            int r = rand() % s;
            int c = rand() % s;
            if ((r != 0 || c != 0) && (r != s - 1 || c != s - 1) && maze[r][c]->getState() == 0) {
                maze[r][c]->setState(1); // Set wall
                wallCount--;
            }
        }

        // Set up directions for the grid
        for (int i = 0; i < s; i++) {
            for (int j = 0; j < s; j++) {
                if (i > 0) maze[i][j]->setDir(UP, maze[i - 1][j]); // Set UP
                if (i < s - 1) maze[i][j]->setDir(DOWN, maze[i + 1][j]); // Set DOWN
                if (j > 0) maze[i][j]->setDir(LEFT, maze[i][j - 1]); // Set LEFT
                if (j < s - 1) maze[i][j]->setDir(RIGHT, maze[i][j + 1]); // Set RIGHT
            }
        }
    }

    List* getPath() {
        List* path = new List();
        bool visited[SIZE][SIZE] = { false };
        if (dfs(0, 0, path, visited)) {
            return path;
        }
    }
}

```

```

        delete path; // No path found
        return new List(); // Return an empty path list
    }

    // DFS function moved here after getPath()
    bool dfs(int x, int y, List* path, bool visited[SIZE][SIZE]) {
        // Check bounds and if the cell is a wall or visited
        if (x < 0 || x >= SIZE || y < 0 || y >= SIZE || maze[x][y]->getState() == 1 || visited[x][y]) {
            return false;
        }

        visited[x][y] = true;
        path->addElement(maze[x][y]);

        // If we reached the bottom-right corner
        if (x == SIZE - 1 && y == SIZE - 1) {
            return true;
        }

        // Explore neighbors
        if (dfs(x + 1, y, path, visited) || // DOWN
            dfs(x - 1, y, path, visited) || // UP
            dfs(x, y + 1, path, visited) || // RIGHT
            dfs(x, y - 1, path, visited)) { // LEFT
            return true;
        }

        path->removeElement(); // Backtrack
        return false;
    }

    void printMaze() {
        for (int i = 0; i < SIZE; i++) {
            for (int j = 0; j < SIZE; j++) {
                cout << maze[i][j]->getState() << " ";
            }
            cout << endl;
        }
    }

    ~Maze() {
        for (int i = 0; i < SIZE; i++) {
            for (int j = 0; j < SIZE; j++) {
                delete maze[i][j]; // Delete each Grid
            }
            delete[] maze[i]; // Delete each row
        }
        delete[] maze; // Delete the array of rows
    }

private:
    Grid*** maze;
};

int main() {
    Maze* maze = new Maze();
    maze->printMaze();
    maze->getPath()->printPath();
    delete maze; // Clean up
}

```

